TITLE: Student Registration Database

ABSTRACTION:

Student Registration System deals with the maintenance of Student Registration data, and students' information within the University. SRS is an automation system, which is used to store the information's, students record, and information of subjects. Starting from registration of a new student in the college, it maintains all the details regarding the Registration of the student.

The project deals with the retrieval of information through an INTRANET based campus wide portal. It collects related information from all the departments of an organization and maintains files, which are used to generate reports in various forms to Registration data of the students. Development process of the system starts with System analysis. System analysis involves creating a formal model of the problem to be solved by understanding requirements.

INTRODUCTION:

The Student Registration system is developed to override the problems prevailing in the practicing manual system, this software is supported to eliminate and, in some cases, reduce the hardships faced by this existing system. Moreover, this system is designed for the need of the University to carry out registration operations in a smooth and effective manner.

The application is reduced as much as possible to avoid errors while entering the data. It also provides error message while entering invalid data. No formal knowledge is needed for the user to use this system. Thus, by this all it proves it is user-friendly. Student Registration System, as described above, can lead to error free, secure, reliable and fast management system. It can assist the user to concentrate on their other activities rather to concentrate on the record keeping. Thus, it will help organization in better utilization of resources.

Every university, whether big or small, has challenges to overcome and managing the information of students, Registrations, Academic Advisor, Professor and also Subjects. This is designed to assist in strategic planning and will help you ensure that the university is equipped with the right level of information and details for your future goals. Also, for those busy executives who are always on the go, our systems come with remote access features, which will allow you to manage your workforce anytime, always. These systems will ultimately allow you to better manage resources.

PURPOSE FOR THE SYSTEM:

The aim of the project is to manage the registration details of students related to major, subjects, professors, as well as the academic advisor. The entire project is built at the administrative end, thus ensuring access to only the administrator. The purpose of the project is to build an application Student Registration Database program to reduce the manual work for managing the Student's Registrations. It tracks all the details about the registration of students and their major, professors, academic advisor.

EXISTING SYSTEM:

In the existing system the registration in computerize way are but if we did it manually it will have some problem like.

- ❖ Lack of security of data.
- ❖ More manpower.
- ❖ Time consuming.
- ❖ Consumes large volume of pare work.

❖ Needs manual calculations.

PROPOSED SYSTEM AND ADVANTAGES OF THE SYSTEM:

The aim of the proposed system is to develop a system of improved Student's registration. The proposed system can overcome all the limitations of the existing system. The system provides proper security and reduces the manual work.

❖ Security of data.

❖ Ensure data accuracy.

❖ Proper control of the higher officials.

❖ Minimize manual data entry.

❖ Minimum time needed for various processing.

❖ Greater efficiency.

❖ Better service.

❖ User friendliness and interactive.

❖ Minimum time required.

REQUIREMENTS COLLECTION AND ANALYSIS:

1. **A student has the following attributes (stu_id a unique value for each record, stu_name, gender, phone_No)**
   One **student** belong to many **registrations** but one **registration** has one **student.**

2. **A major has the following attributes (m_name a unique value for each record, years, levels)**
   One **major** on many **registrations** but one **registration** has one **major**.

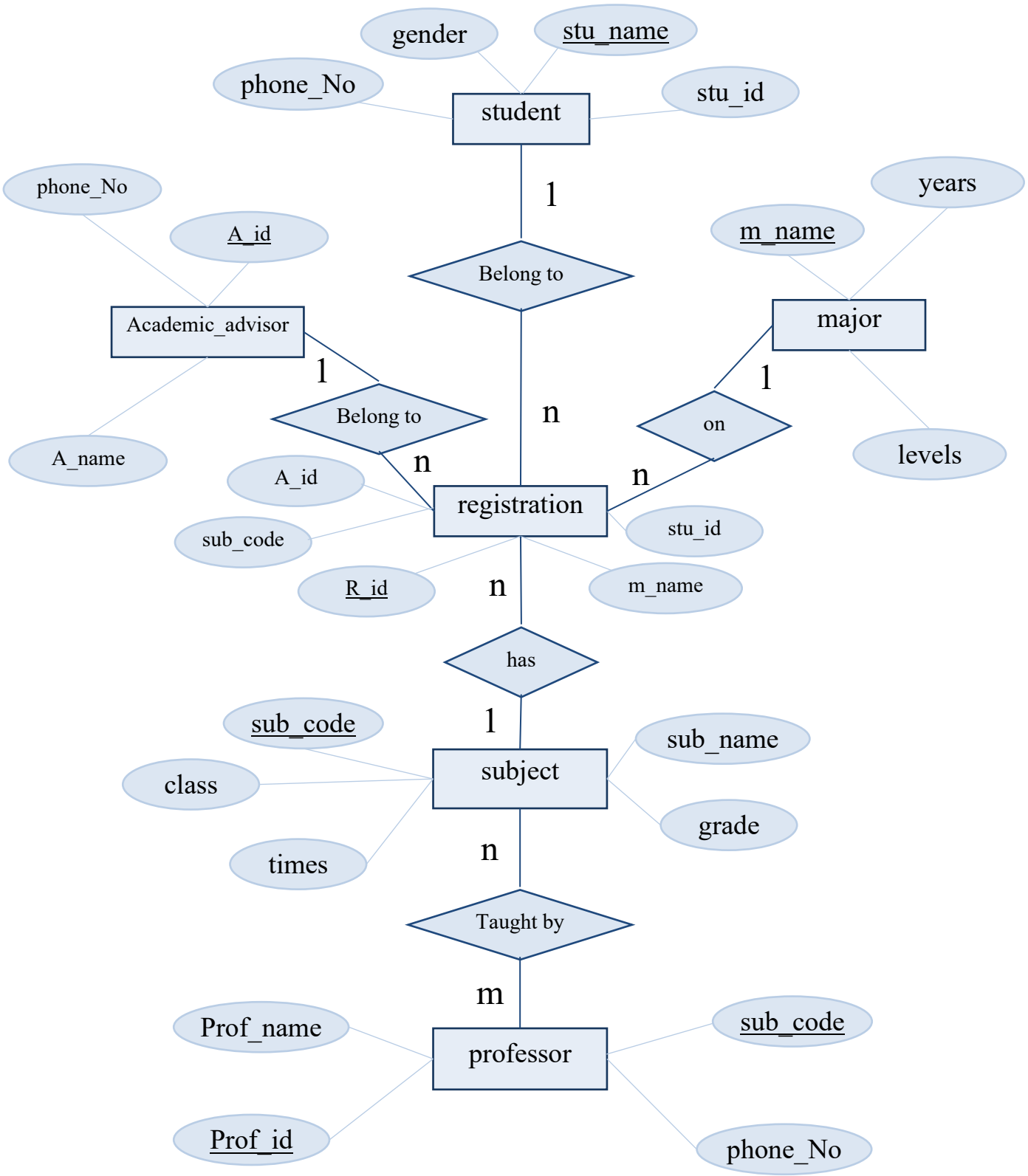
3. **A subject has the following attributes (sub_code a unique value for each record, sub_name, class, times, grade)**
   One **subject** has many **registrations** but one **registration** has one **subject**.

4. **A academic advisor has the following attributes (A_name, A_id a unique value for each record, phone_No)**
   One **academic advisor** belong to many **registrations** but one **registration** has one **academic advisor**.

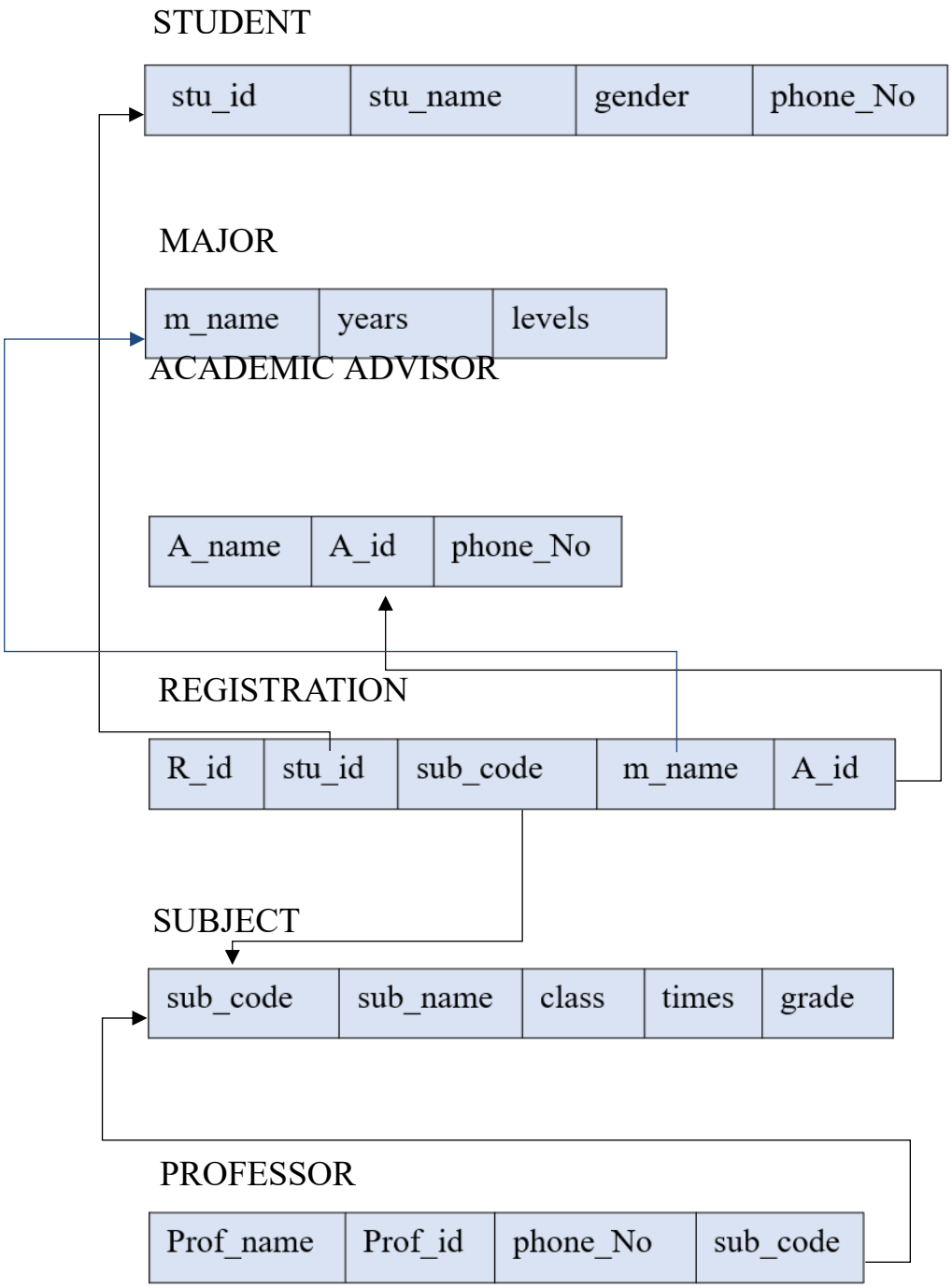
5. **A professor has the following attributes (Prof_name, Prof_id, phone_No, sub_code)**
   One **professor** has many **subjects** and one **subject** Taught by many **professors**.

6. **A registration has the following attributes (R_id, stu_id, sub_code, m_name, A_id)**
   One **registration** has one: **academic advisor**, **subject, major, student**.

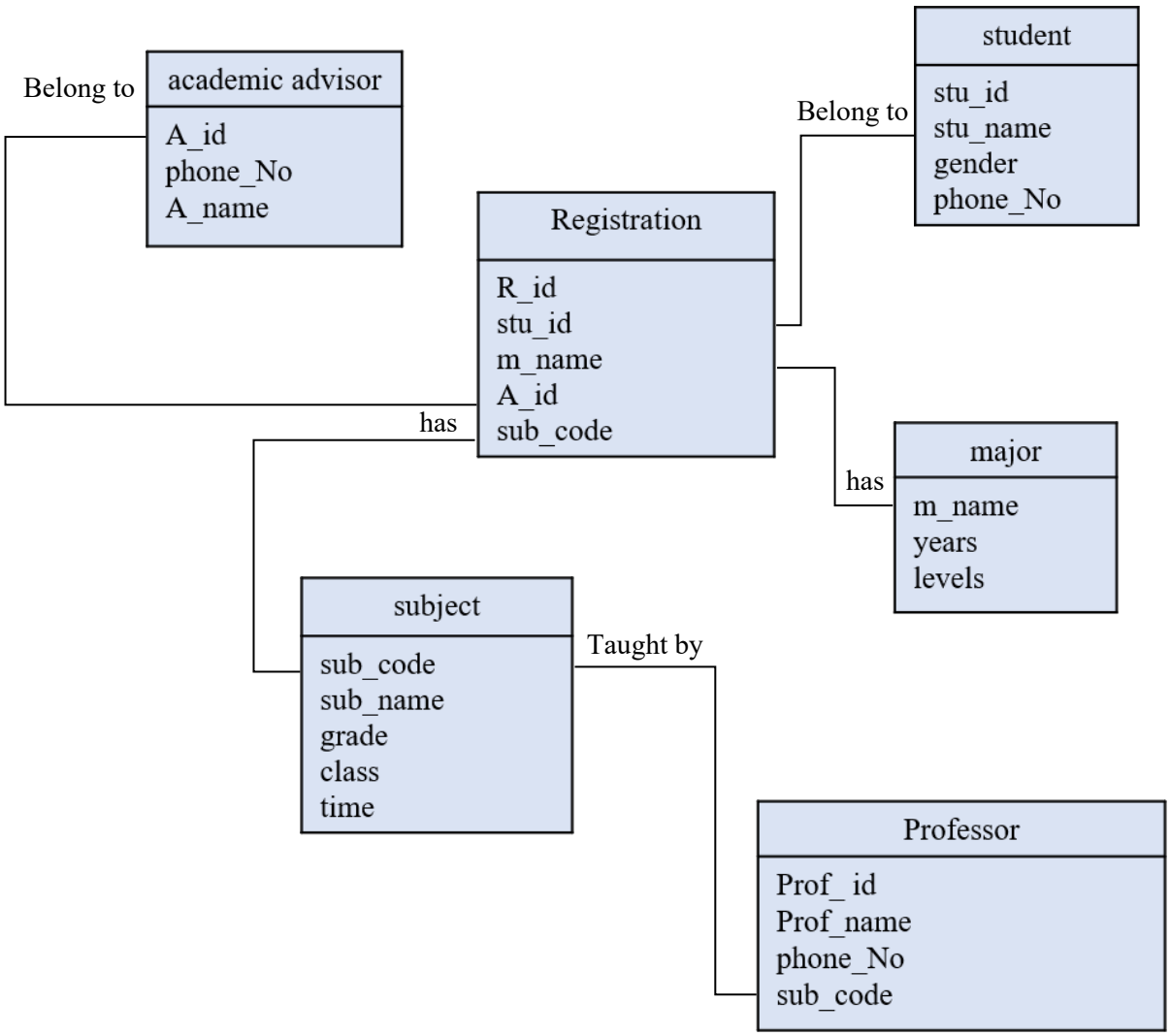
CONCEPTUAL DESIGN: E-R DIAGRAM:



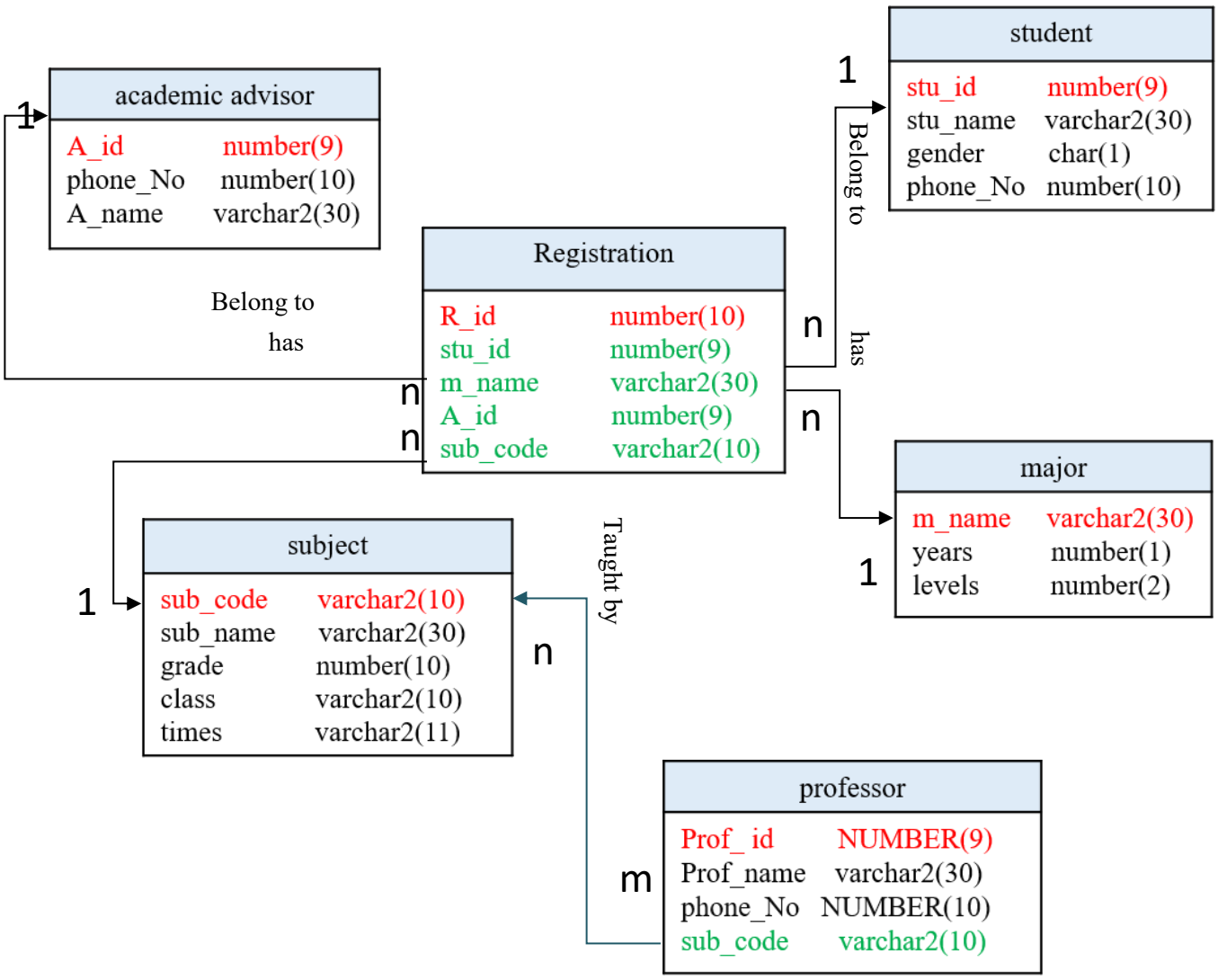
LOGICAL SCHEMA DESIGN MAPPIN:



PHYSICAL SCHEMA DESIGN-INTERNAL SCHEMA:



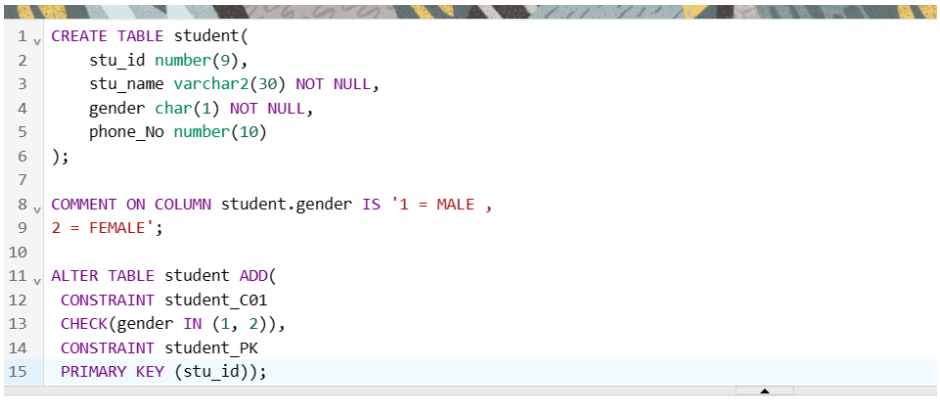
LOGICAL AND PHYSICAL MODEL DESIGN (SCHEMA DESIGN MAPPING):



QUERY FOR CREATING STUDENT REGISTRATION DATABASE:

1. STUDENT

```
CREATE TABLE student(
   stu_id number(9),
        stu_name varchar2(30) NOT NULL,
        gender char(1) NOT NULL,
        phone_No number(10)
);

COMMENT ON COLUMN student.gender IS '1 = MALE ,
2 = FEMALE';

ALTER TABLE student ADD(
 CONSTRAINT student_C01
 CHECK(gender IN (1, 2)),
 CONSTRAINT student_PK
 PRIMARY KEY (stu_id));
```



```
CREATE TABLE student(
    stu_id number(9),
    stu_name varchar2(30) NOT NULL,
    gender char(1) NOT NULL,
    phone_No number(10)
);

COMMENT ON COLUMN student.gender IS '1 = MALE ,
2 = FEMALE';

ALTER TABLE student ADD(
 CONSTRAINT student_C01
 CHECK(gender IN (1, 2)),
 CONSTRAINT student_PK
 PRIMARY KEY (stu_id));
```

```
Table created.

Statement processed.

Table altered.
```

2. MAJOR

```
CREATE TABLE major
(
m_name     varchar2(30),
years      number(1),
levels     number(2)
);
```

ALTER TABLE major ADD (

CONSTRAINT major_PK

PRIMARY KEY (m_name));

```
CREATE TABLE major
(
m_name      varchar2(30),
years       number(1),
levels      number(2)
);
 ALTER TABLE major ADD (
 CONSTRAINT major_PK
 PRIMARY KEY (m_name));
```

```
Table created.

Table altered.
```

3. SUBJECT

CREATE TABLE subject
(
sub_code   varchar2(10),
sub_name   varchar2(30)NOT NULL,
grade   number(10),
class   varchar2(10),
times   varchar2(11)
);
ALTER TABLE subject ADD (
 CONSTRAINT subject_PK
 PRIMARY KEY (sub_code));

```
CREATE TABLE subject
(
sub_code   varchar2(10),
sub_name   varchar2(30)NOT NULL,
grade      number(10),
class      varchar2(10),
times      varchar2(11)
);
ALTER TABLE subject ADD (
 CONSTRAINT subject_PK
 PRIMARY KEY (sub_code));
```

```
Table created.

Table altered.
```

4. ACADEMIC ADVISOR

CREATE TABLE academic_advisor
(
 A_id   number(9),
phone_No number(10) NOT NULL,
A_name   varchar2(30)
);

ALTER TABLE academic_advisor ADD (
CONSTRAINT academic_dvisor_PK
PRIMARY KEY (A_id));

```
CREATE TABLE academic_advisor
(
 A_id      number(9),
phone_No   number(10) NOT NULL,
A_name     varchar2(30)
);
ALTER TABLE academic_advisor ADD
CONSTRAINT academic_dvisor_PK
PRIMARY KEY ( A_id ));

```

```
Table created.

Table altered.
```

5. PROFISSIOR

CREATE TABLE professor
(
Prof_id   NUMBER(9) NOT NULL,
Prof_name varchar2(30),
phone_No   NUMBER(10),
sub_code   varchar2(10)
);

ALTER TABLE professor ADD (
 CONSTRAINT professor_PK
 PRIMARY KEY (Prof_id));

ALTER TABLE professor ADD(
 CONSTRAINT professor_R01
 FOREIGN KEY(sub_code)
 REFERENCES subject (sub_code));

```
CREATE TABLE professor
(
Prof_id   NUMBER(9) NOT NULL,
Prof_name  varchar2(30),
phone_No   NUMBER(10),
sub_code   varchar2(10)
);

ALTER TABLE professor ADD (
 CONSTRAINT professor_PK
 PRIMARY KEY (Prof_id));

ALTER TABLE professor ADD(
 CONSTRAINT professor_R01
 FOREIGN KEY(sub_code)
 REFERENCES subject (sub_code));

```

```
Table created.

Table altered.

Table altered.
```

6. REGISTRATION

```
CREATE TABLE Registration(
   R_id  number(10),
    stu_id  number(9),
    m_name varchar2(30),
    A_id number(9),
    sub_code varchar2(10));

ALTER TABLE Registration ADD(
 CONSTRAINT Registration_PK
 PRIMARY KEY (R_id));

ALTER TABLE Registration ADD(
   CONSTRAINT Registration_R01
 FOREIGN KEY(stu_id)
 REFERENCES student (stu_id),
   CONSTRAINT Registration_R02
 FOREIGN KEY(m_name)
 REFERENCES major (m_name),
   CONSTRAINT Registration_R03
 FOREIGN KEY(A_id)
 REFERENCES academic_advisor  (A_id),
   CONSTRAINT Registration_R04
 FOREIGN KEY(sub_code)
 REFERENCES subject (sub_code));
```

```
CREATE TABLE Registration(
    R_id  number(10),
    stu_id  number(9),
    m_name varchar2(30),
    A_id number(9),
    sub_code varchar2(10));

ALTER TABLE Registration ADD(
 CONSTRAINT Registration_PK
 PRIMARY KEY (R_id));

ALTER TABLE Registration ADD(
    CONSTRAINT Registration_R01
 FOREIGN KEY(stu_id)
 REFERENCES student (stu_id),
    CONSTRAINT Registration_R02
 FOREIGN KEY(m_name)
 REFERENCES major (m_name),
    CONSTRAINT Registration_R03
 FOREIGN KEY(A_id)
 REFERENCES academic_advisor  (A_id),
    CONSTRAINT Registration_R04
 FOREIGN KEY(sub_code)
 REFERENCES subject (sub_code));
```

```
Table created.

Table altered.

Table altered.
```

INSERT DATA TO STUDENT REGISTRATION DATABASE BY USING PROCEDUR:

In order to insert data into table, we first need to create procedure and parameters. Then, we call that procedure and put the right data into parameter as following:

1. STUDENT_INSERT

```
CREATE OR REPLACE PROCEDURE student_insert
  (
  P_stu_id    student.stu_id%TYPE,
  P_stu_name  student.stu_name%TYPE,
  P_gender    student.gender%TYPE,
  P_phone_No  student.phone_No%TYPE
  )
  AS
  BEGIN
  INSERT INTO student (
    stu_id,
                stu_name,
                gender,
                phone_No)
  VALUES(
    P_stu_id,
      P_stu_name,
      P_gender,
      P_phone_No);
      COMMIT;
  DBMS_OUTPUT.PUT_LINE('RECORD OF ' ||P_stu_name||'
```

```
    '||'INSERTED SUCCESSFULLY ....!');
    EXCEPTION
    WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERRORS HAPPENED '||'
    '||SQLERRM);
END;
/
```

```
CREATE OR REPLACE PROCEDURE student_insert
    (
    P_stu_id    student.stu_id%TYPE,
    P_stu_name  student.stu_name%TYPE,
    P_gender    student.gender%TYPE,
    P_phone_No  student.phone_No%TYPE
    )
    AS
    BEGIN
    INSERT INTO student (
        stu_id,
        stu_name,
        gender,
        phone_No)
    VALUES(
        P_stu_id,
        P_stu_name,
        P_gender,
        P_phone_No);
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('RECORD OF ' ||P_stu_name||'
    '||'INSERTED SUCCESSFULLY ....!');
    EXCEPTION
    WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERRORS HAPPENED '||'
    '||SQLERRM);
END;
/
```

```
Procedure created.
```

CALL PROCRDURE

EXEC student_insert (1,'WIJDAN', 2, 0555436789);

```
EXEC student_insert (1,'WIJDAN', 2, 0555436789);
```

```
Statement processed.
RECORD OF WIJDAN
    INSERTED SUCCESSFULLY ....!
```

EXEC student_insert (2,'RAGHAD', 2, 0554398789);

```
1  EXEC student_insert (2,'RAGHAD', 2, 0554398789);
```

```
Statement processed.
RECORD OF RAGHAD
    INSERTED SUCCESSFULLY ....!
```

SELECT STATEMENT

SELECT * FROM student;

```
1  SELECT * FROM student;
```

STU_ID	STU_NAME	GENDER	PHONE_NO
2	RAGHAD	2	554398789
1	WIJDAN	2	555436789

Download CSV

2 rows selected.

2. MAJOR_INSERT

```
CREATE OR REPLACE PROCEDURE major_INSERT
(
P_m_name      major.m_name%TYPE,
P_years     major.years%TYPE,
P_levels    major.levels%TYPE
   )

AS
BEGIN
INSERT INTO major (m_name,years,levels)

VALUES (P_m_name,P_years,P_levels);

COMMIT;

DBMS_OUTPUT.PUT_LINE('RECORD OF ' ||P_m_name||'
'||'INSERTED SUCCESSFULLY ....!');
EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERRORS HAPPENED '||'
'||SQLERRM);
END;
/
```

```
CREATE OR REPLACE PROCEDURE major_INSERT
(
P_m_name      major.m_name%TYPE,
P_years     major.years%TYPE,
P_levels    major.levels%TYPE
    )

AS
BEGIN
INSERT INTO major (m_name,years,levels)

VALUES (P_m_name,P_years,P_levels);

COMMIT;

DBMS_OUTPUT.PUT_LINE('RECORD OF ' ||P_m_name||'
'||'INSERTED SUCCESSFULLY ....!');
EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERRORS HAPPENED '||'
'||SQLERRM);
END;
/
```

```
Procedure created.
```

CALL PROCRDURE

EXEC major_INSERT ('computer Science',5,15);
EXEC major_INSERT ('Physics',4,12);
EXEC major_INSERT ('Mathematic',4,12);

```
EXEC major_INSERT  ('computer Science',5,15);
EXEC major_INSERT  ('Physics',4,12);
EXEC major_INSERT  ('Mathematic',4,12);
```

```
Statement processed.
RECORD OF computer Science
INSERTED SUCCESSFULLY ....!

Statement processed.
RECORD OF Physics
INSERTED SUCCESSFULLY ....!

Statement processed.
RECORD OF Mathematic
INSERTED SUCCESSFULLY ....!
```

SELECT STATEMENT

SELECT * FROM major;

```
1  SELECT * FROM major;
```

M_NAME	YEARS	LEVELS
computer Science	5	15
Physics	4	12
Mathematic	4	12

Download CSV

3 rows selected.

3. SUBJECT_INSERT

```
CREATE OR REPLACE PROCEDURE subject_INSERT   (
P_sub_code      subject.sub_code%TYPE,
P_sub_name      subject.sub_name%TYPE,
P_grade         subject.grade %TYPE,
P_class         subject.class%TYPE,
P_times         subject.times %TYPE   )
AS
BEGIN
INSERT INTO subject
   (  sub_code,  sub_name,   grade,   class,   times   )
VALUES (
 P_sub_code,
 P_sub_name,
 P_grade,
 P_class,
 P_times   );
COMMIT;
DBMS_OUTPUT.PUT_LINE('RECORD OF ' ||P_sub_name||'
'||'INSERTED SUCCESSFULLY ....!');
EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERRORS HAPPENED '||'
'||SQLERRM);
```

END;

/

```
CREATE OR REPLACE PROCEDURE subject_INSERT   (
P_sub_code      subject.sub_code%TYPE,
P_sub_name      subject.sub_name%TYPE,
P_grade         subject.grade %TYPE,
P_class         subject.class%TYPE,
P_times         subject.times %TYPE   )
AS
BEGIN
INSERT INTO subject
    (   sub_code,   sub_name,    grade,    class,    times   )
VALUES (
 P_sub_code,
 P_sub_name,
 P_grade,
 P_class,
 P_times   );
COMMIT;
DBMS_OUTPUT.PUT_LINE('RECORD OF ' ||P_sub_name||'
'||'INSERTED SUCCESSFULLY ....!');
EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERRORS HAPPENED '||'
'||SQLERRM);
END;
/
```

```
Procedure created.
```

CALL PROCRDURE

EXEC subject_INSERT('Math','Algebra',90,'ES-15','12:00');

EXEC subject_INSERT('DB','DBMS',99,'BS-05','9:00');

```
EXEC subject_INSERT('Math','Algebra',90,'ES-15','12:00');
EXEC subject_INSERT('DB','DBMS',99,'BS-05','9:00');
```

```
Statement processed.
RECORD OF Algebra
INSERTED SUCCESSFULLY ....!

Statement processed.
RECORD OF DBMS
INSERTED SUCCESSFULLY ....!
```

SELECT STATEMENT

SELECT * FROM subject;

```
1  SELECT * FROM subject;
2
```

SUB_CODE	SUB_NAME	GRADE	CLASS	TIMES
Math	Algebra	90	ES-15	12:00
DB	DBMS	99	BS-05	9:00

Download CSV

2 rows selected.

4. ACADEMIC _ADVISOR_INSERT

```
CREATE OR REPLACE PROCEDURE academic_advisor_INSERT
(
P_A_id       academic_advisor.A_id%TYPE,
P_A_name      academic_advisor.A_name%TYPE,
P_phone_No    academic_advisor.phone_No %TYPE
)
AS
BEGIN
INSERT INTO academic_advisor(
A_id,
A_name,
phone_No
 )
VALUES (
P_A_id,
P_A_name,
P_phone_No
);
```

COMMIT;

DBMS_OUTPUT.PUT_LINE('RECORD OF ' ||P_A_name||'

'||'INSERTED SUCCESSFULLY ....!');

EXCEPTION

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERRORS HAPPENED '||'

'||SQLERRM);

END;

/

```
CREATE OR REPLACE PROCEDURE academic_advisor_INSERT
(
P_A_id        academic_advisor.A_id%TYPE,
P_A_name       academic_advisor.A_name%TYPE,
P_phone_No     academic_advisor.phone_No %TYPE
)
AS
BEGIN
INSERT INTO academic_advisor(
A_id,
A_name,
phone_No
 )
VALUES (
P_A_id,
P_A_name,
P_phone_No
);
COMMIT;
DBMS_OUTPUT.PUT_LINE('RECORD OF ' ||P_A_name||'
'||'INSERTED SUCCESSFULLY ....!');
EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERRORS HAPPENED '||'
'||SQLERRM);
END;
/
```

```
Procedure created.
```

CALL PROCRDURE

EXEC academic_advisor_INSERT(1,'HIND', 0566677754);
EXEC academic_advisor_INSERT(2,'LOJIAN', 0566337754);

```
1  EXEC academic_advisor_INSERT(1,'HIND', 0566677754);
2  EXEC academic_advisor_INSERT(2,'LOJIAN', 0566337754);
```

```
Statement processed.
RECORD OF HIND
INSERTED SUCCESSFULLY ....!

Statement processed.
RECORD OF LOJIAN
INSERTED SUCCESSFULLY ....!
```

SELECT STATEMENT

SELECT * FROM academic_advisor;

```
1  SELECT * FROM academic_advisor;
```

A_ID	PHONE_NO	A_NAME
1	566677754	HIND
2	566337754	LOJIAN

Download CSV

2 rows selected.

5. PROFESSOR_INSERT

```
CREATE OR REPLACE PROCEDURE professor_INSERT
(
P_Prof_id professor.Prof_id%TYPE,
P_Prof_name professor.Prof_name%TYPE,
P_phone_No professor.phone_No%TYPE,
P_sub_code professor.sub_code%TYPE)
AS
BEGIN
INSERT INTO professor (Prof_id, Prof_name,phone_No,sub_code)
VALUES (P_Prof_id,P_Prof_name,P_phone_No,P_sub_code);
COMMIT;
DBMS_OUTPUT.PUT_LINE('RECORD OF ' ||P_Prof_name||'
'||'INSERTED SUCCESSFULLY ....!');
EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERRORS HAPPENED '||'
'||SQLERRM);
```

END;

```
CREATE OR REPLACE PROCEDURE professor_INSERT
(
P_Prof_id professor.Prof_id%TYPE,
P_Prof_name professor.Prof_name%TYPE,
P_phone_No professor.phone_No%TYPE,
P_sub_code professor.sub_code%TYPE)
AS
BEGIN
INSERT INTO professor (Prof_id, Prof_name,phone_No,sub_code)
VALUES (P_Prof_id,P_Prof_name,P_phone_No,P_sub_code);
COMMIT;
DBMS_OUTPUT.PUT_LINE('RECORD OF ' ||P_Prof_name||'
'||'INSERTED SUCCESSFULLY ....!');
EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERRORS HAPPENED '||'
'||SQLERRM);
END;
/
```

```
Procedure created.
```

/

CALL PROCRDURE

EXEC professor_INSERT(124567535,'Aisha ALQahtani',0554433221,'DB');
EXEC professor_INSERT(864987535,'Aisha Muhammad',0987433221,'Math');

```
EXEC professor_INSERT(124567535,'Aisha ALQahtani',0554433221,'DB');
EXEC professor_INSERT(864987535,'Aisha Muhammad',0987433221,'Math');
```

```
Statement processed.
RECORD OF Aisha ALQahtani
INSERTED SUCCESSFULLY ....!

Statement processed.
RECORD OF Aisha Muhammad
INSERTED SUCCESSFULLY ....!
```

SELECT STATEMENT

SELECT * FROM professor;

```
1  SELECT * FROM professor;
```

PROF_ID	PROF_NAME	PHONE_NO	SUB_CODE
124567535	Aisha ALQahtani	554433221	DB
864987535	Aisha Muhammad	987433221	Math

Download CSV

2 rows selected.

6. REGISTRATION_INSERT

```
CREATE OR REPLACE PROCEDURE Registration_INSERT(
P_R_ID Registration.R_ID%TYPE,
P_stu_id Registration.stu_id%TYPE,
P_m_name Registration.m_name%TYPE,
P_A_id  Registration.A_id %TYPE,
P_sub_code Registration.sub_code%TYPE)
AS
BEGIN
INSERT INTO Registration(
R_ID,
stu_id,
m_name,
A_id,
sub_code)
VALUES(
 P_R_ID,
 P_stu_id,
 P_m_name,
 P_A_id,
 P_sub_code);
 COMMIT;
DBMS_OUTPUT.PUT_LINE('RECORD OF R_ID ' ||P_R_ID||'
'||'INSERTED SUCCESSFULLY ....!');
EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERRORS HAPPENED '||' '||SQLERRM);
END;
/
```

```
1  CREATE OR REPLACE PROCEDURE Registration_INSERT(
2  P_R_ID Registration.R_ID%TYPE,
3  P_stu_id Registration.stu_id%TYPE,
4  P_m_name Registration.m_name%TYPE,
5  P_A_id  Registration.A_id %TYPE,
6  P_sub_code Registration.sub_code%TYPE)
7  AS
8  BEGIN
9  INSERT INTO Registration(
10 R_ID,
11 stu_id,
12 m_name,
13 A_id,
14 sub_code)
15 VALUES(
16  P_R_ID,
17  P_stu_id,
18  P_m_name,
19  P_A_id,
20  P_sub_code);
21  COMMIT;
22 DBMS_OUTPUT.PUT_LINE('RECORD OF R_ID ' ||P_R_ID||'
23 '||'INSERTED SUCCESSFULLY ....!');
24 EXCEPTION
25 WHEN OTHERS THEN
26 DBMS_OUTPUT.PUT_LINE('UNEXPECTED ERRORS HAPPENED '||' '||SQLERRM);
27 END;
28 /
```

```
Procedure created.
```

CALL PROCRDURE

EXEC Registration_INSERT(1, 1, 'computer Science', 1,'DB');

EXEC Registration_INSERT(2, 2, 'Mathematic', 2,'Math');

```
1  EXEC Registration_INSERT(1, 1, 'computer Science', 1,'DB');
2  EXEC Registration_INSERT(2, 2, 'Mathematic', 2,'Math');
```

```
Statement processed.
RECORD OF R_ID 1
INSERTED SUCCESSFULLY ....!

Statement processed.
RECORD OF R_ID 2
INSERTED SUCCESSFULLY ....!
```

SELECT STATEMENT

SELECT * FROM Registration;

```
1  SELECT * FROM Registration;
```

R_ID	STU_ID	M_NAME	A_ID	SUB_CODE
1	1	computer Science	1	DB
2	2	Mathematic	2	Math

Download CSV

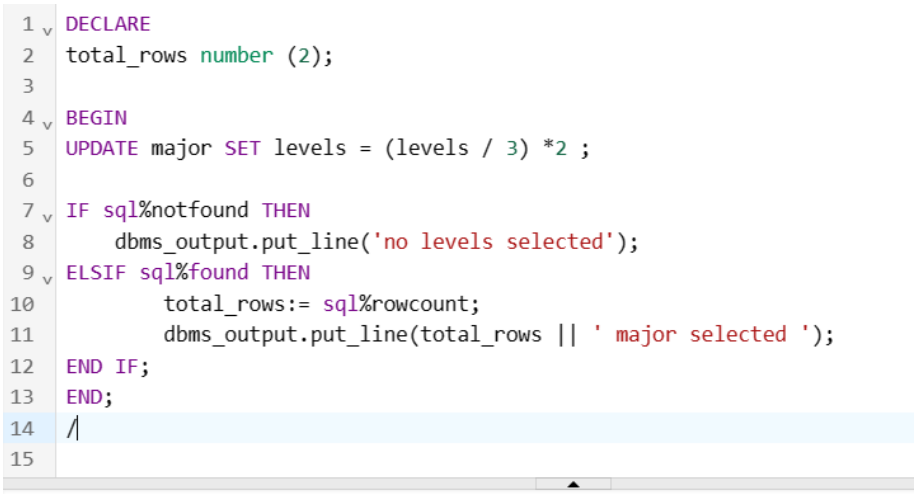
2 rows selected.

IMPLEMENTATION CURSORS USING PL/SQL-DECLARE BEGIN END TO UPDATE DATA:

1. UPDATING THE NUMBER OF LEVES TO BE FOR 2 SEMESTER:

DECLARE
total_rows number (2);

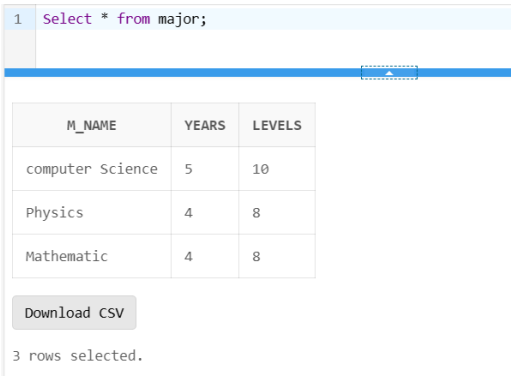
```
BEGIN
UPDATE major SET levels = (levels / 3) *2 ;

IF sql%notfound THEN
        dbms_output.put_line('no levels selected');
ELSIF sql%found THEN
        total_rows:= sql%rowcount;
        dbms_output.put_line(total_rows || ' major selected ');
END IF;
END;
/
```




```
Statement processed.
3 major selected
```

SELECT STATEMENT

Select * from major;



2. UPDATING THE PROFESSOR NAME BY ADDING "Prof." BEFORE THE NAME:

```
DECLARE
total_rows number (2);

BEGIN
UPDATE professor SET Prof_name = 'Prof.' || Prof_name ;

IF sql%notfound THEN
        dbms_output.put_line('no professor are selected');
ELSIF sql%found THEN
```

```
        total_rows:= sql%rowcount;
        dbms_output.put_line(total_rows || ' professor  selected ');
END IF;

END;
/
```

```
DECLARE
total_rows number (2);

BEGIN
UPDATE professor SET Prof_name = 'Prof.' || Prof_name ;

IF sql%notfound THEN
    dbms_output.put_line('no professor are selected');
ELSIF sql%found THEN
    total_rows:= sql%rowcount;
    dbms_output.put_line(total_rows || ' professor  selected ');
END IF;

END;
/
```

```
Statement processed.
2 professor  selected
```

SELECT STATEMENT

select * from professor;

```
select * from professor;
```

PROF_ID	PROF_NAME	PHONE_NO	SUB_CODE
124567535	Prof.Aisha ALQahtani	554433221	DB
864987535	Prof.Aisha Muhammad	987433221	Math

Download CSV

2 rows selected.